Submission Worksheet

Submission Data

Course: IT114-007-F2025
Assignment: IT114 - Milestone 3 - Hangman
Student: Evans M. (ekn3)
Status: Submitted | **Worksheet Progress:** 100%
Potential Grade: 10.00/10.00 (100.00%)
Received Grade: 0.00/10.00 (0.00%)
Started: 12/7/2025 10:28:12 PM
Updated: 12/8/2025 10:56:04 PM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT114-007-F2025/it114-milestone-3-hangman/grading/ekn3
View Link: https://learn.ethereallab.app/assignment/v3/IT114-007-F2025/it114-milestone-3-hangman/view/ekn3

Instructions

1. Refer to Milestone3 of Hangman / Word guess
    1. Complete the features
2. Ensure all code snippets include your ucid, date, and a brief description of what the code does
3. Switch to the `Milestone3` branch
    1. `git checkout Milestone3`
    2. `git pull origin Milestone3`
4. Fill out the below worksheet as you test/demo with 3+ clients in the same session
5. Once finished, click "Submit and Export"
6. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin Milestone3`
    4. On Github merge the pull request from `Milestone3` to `main`
7. Upload the same PDF to Canvas
8. Sync Local
    1. `git checkout main`
    2. `git pull origin main`

Section #1: (1 pt.) Core Ui

Progress: 100%

Task #1 (0.50 pts.) - Connection/Details Panels

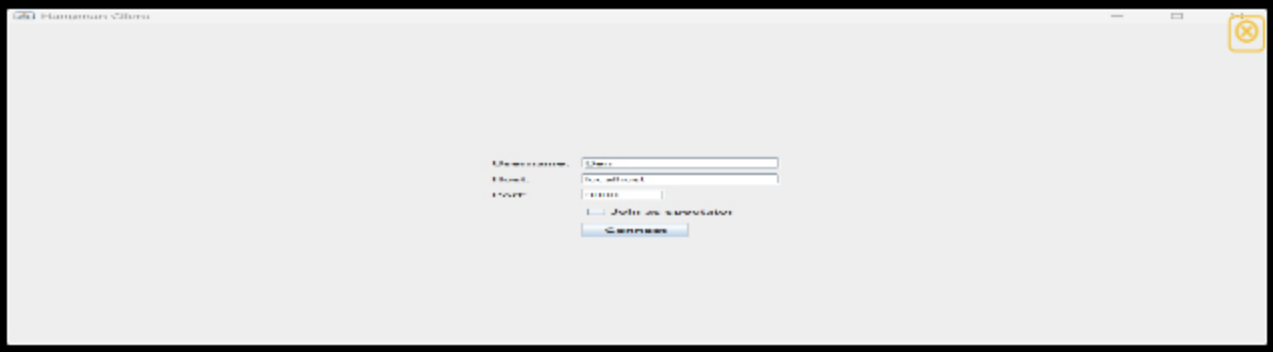
Progress: 100%

Part 1:

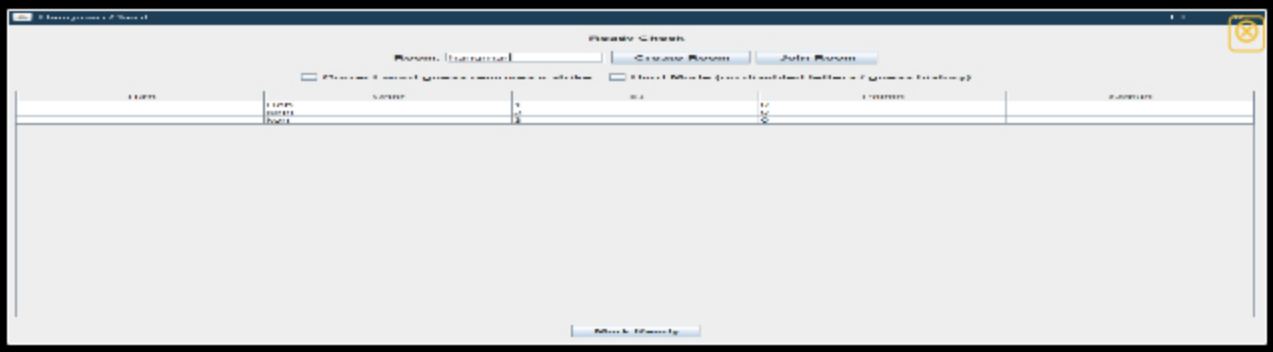
Progress: 100%

Details:

- Show the connection panel with valid data
- Show the user details panel with valid data



Connection Panel



User panel

Saved: 12/8/2025 2:14:58 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow from recording/capturing these details and passing them through the connection process

Your Response:

The ConnectPanel collects user inputs then Client sends them to the server. The server registers the user in a room. BaseGameRoom synchronizes room/player state to the new client and the client UI displays all user details using GamePanel/ReadyPanel tables.

Saved: 12/8/2025 2:14:58 PM

Task #2 (0.50 pts.) - Ready Panel

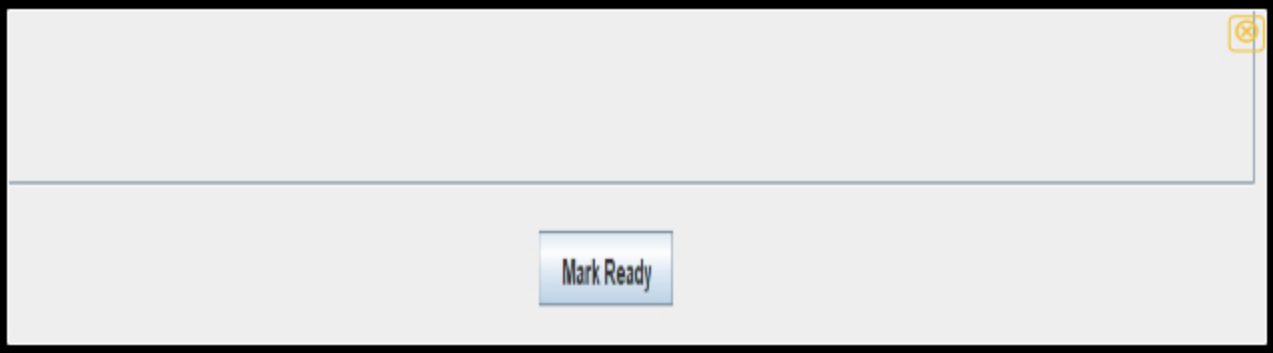
Progress: 100%

Part 1:

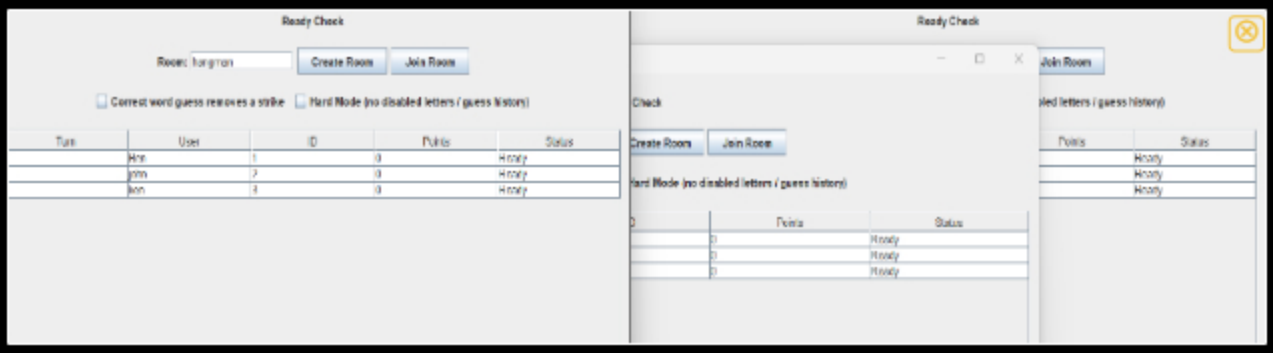
Progress: 100%

Details:

- Show the button used to mark ready
- Show a few variations of indicators of clients being ready (3+ clients)



mark ready button



variations

Saved: 12/8/2025 2:28:56 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for marking READY from the UI
- Briefly explain the code flow from receiving READY data and updating the UI

Your Response:

User marks ready from the UI, server sends ready payload and the server sets ready flag and broadcast u[pdate the client recieves the ready payload , updates the user object and the Ui refreshes table and shows updated ready staus

Saved: 12/8/2025 2:28:56 PM

Section #2: (2 pts.) Project Ui

Progress: 100%

Task #1 (0.67 pts.) - User List Panel

Progress: 100%

Details:

- Show the username and id of each user
- Show the current points of each user
- Users should appear in turn order
- Show an indicator of whose turn it is

Part 1:

Progress: 100%

Details:

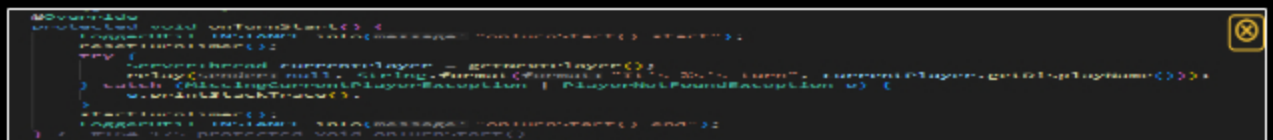
- Show various examples of points (3+ clients visible)
  - Include code snippets showing the code flow for this from server-side to UI
- Show that the sorting is maintained across clients
  - Include code snippets showing the code that handles this
- Show various examples of the turn indicators
  - Include code snippets showing the code flow for this from server-side to UI

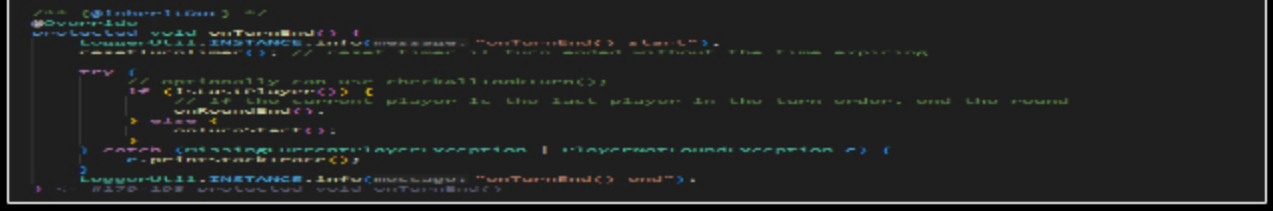


code flow snippet from server-side to UI

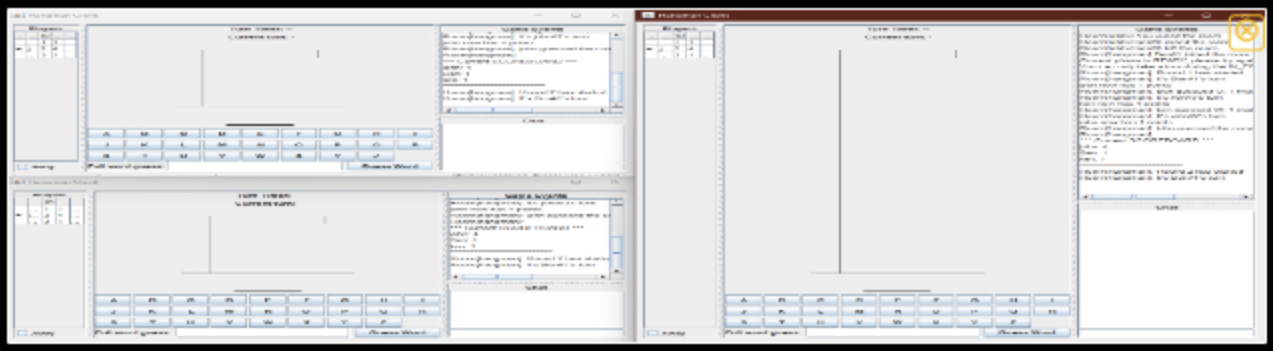
```
private void syncScoreboard(boolean isFinal) {
    List<SeverUser> sorted = users.values().stream()
            .sorted((a, b) -> Integer.compare(b.getPoints(), a.getPoints()))
            .collect(Collectors.toList());
    StringBuilder sb = new StringBuilder(isFinal ? "\n*** FINAL SCOREBOARD ***\n" : "\n*** Current SCOREBOARD ***\n");
    sorted.forEach(p -> sb.append(String.format(format: "%s: %d\n", p.getClientName(), p.getPoints())));
    sb.append(str: "**************************");
    relay(sender: null, sb.toString());
} <- #396-404 private void syncScoreboard(boolean isFinal)
```

sorting code snippet





turn indicators



username and ID of each user

```
wordGuess = wordGuess.toUpperCase();
if (wordGuess.equals(currentWord)) {
    int missingLetters = (int) new String(currentBlanks).chars().filter(c -> c == '_').count();
    int pointsEarned = missingLetters * 2;
    guesser.setPoints(guesser.getPoints() + pointsEarned);

    // UPDATE: optional config - remove one strike on correct guess
    if (removeStrikeOnCorrectGuess && strikes > 0) {
        strikes--;
    }

    // sync points to all clients
    clientsInRoom.values().forEach(c -> c.sendPoints(guesser.getClientId(), guesser.getPoints()));

    relay(sender: null, String.format(format: "%s guessed the correct word %s and got %d points",
            guesser.getClientName(), currentWord, pointsEarned));

    // reveal word and end round
    currentBlanks = currentWord.toCharArray();
    syncBlanksAndStrikes();
    onRoundEnd();
} else {
```

code flow snippet from server-side to UI

```
private void processPoints(Payload payload) {
    if (payload instanceof PointsPayload) {
        PointsPayload pp = (PointsPayload) payload;
        if (knownClients.containsKey(pp.getClientId())) {
            knownClients.get(pp.getClientId()).setPoints(pp.getPoints());
            LoggerUtil.INSTANCE.info(TextFX.colorize(String.format(format: "Client %s now has %d points",
                    pp.getClientId(), pp.getPoints()), Color.YELLOW));
        } <- #626-630 if (knownClients.containsKey(pp.getClientId()))
    } <- #624-631 if (payload instanceof PointsPayload)
} <- #623-632 private void processPoints(Payload payload)
```

code flow snippet from server-side to UI

```
for (User u : users) {        You, 2 days ago • Milestone3 UI
    int newPoints = u.getPoints();
    Integer oldPoints = lastPointsSeen.get(u.getClientId());
    if (oldPoints == null) {
        lastPointsSeen.put(u.getClientId(), newPoints);
    } else if (oldPoints != newPoints) {
        lastPointsSeen.put(u.getClientId(), newPoints);
        gamePanel.appendEvent(
                String.format(format: "%s now has %d points", u.getClientName(), newPoints));
    } <- #146-150 else if (oldPoints != newPoints)
} <- #141-151 for (User u : users)
```

code flow snippet from server-side to UI

```
Downloads/IT 114/New folder/Ekn3-IT114-
7 (Milestone3)
$ java Projects.Client.HangmanClientUI
*** Current SCOREBOARD ***
Carl1: 1
John2: 0
Amy3: 0
Ben4: 0
*****************************
```

sorting display

```
} <- #247-254 private void sendResetTurnStatus()

private void sendTurnStatus(ServerThread client, boolean tookTurn) {      You, 3 weeks ago via  PR #33 • Turn based code
    clientsInRoom.values().removeIf(spInRoom -> {
        boolean failedToSend = !spInRoom.sendTurnStatus(client.getClientId(), client.didTakeTurn());
        if (failedToSend) {
            removeClient(spInRoom);
        }
        return failedToSend;
    }); <- #257-263 clientsInRoom.values().removeIf
```

turn indicators

```
// Note: For now assuming ReadyPayload (this may be changed later)
if (!(payload instanceof ReadyPayload)) {
    error(message: "invalid payload subclass for processTurn");
    return;
}
ReadyPayload rp = (ReadyPayload) payload;
if (!knownClients.containsKey(rp.getClientId())) {
    LoggerUtil.INSTANCE.severe(String.format(format: "Received turn status for client id %s who is not known",
            rp.getClientId()));
    return;
} <- #664-667 if (!knownClients.containsKey(rp.getClientId()))
User cp = knownClients.get(rp.getClientId());
cp.setTookTurn(rp.isReady());

// UI: track current turn client
currentTurnClientId = cp.getClientId();
lastTurnMessage = rp.getMessage();

if (payload.getPayloadType() != PayloadType.SYNC_TURN) {
    String message = String.format(format: "%s %s their turn", cp.getDisplayName(),
            cp.didTakeTurn() ? "took" : "reset");
    LoggerUtil.INSTANCE.info(message);
} <- #669-673 if (payload.getPayloadType() != PayloadType.SYNC_TURN)
} <- #650-674 private void processTurn(Payload payload)
```

turun indicators

Saved: 12/8/2025 3:27:15 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for points updates from server-side to the UI
- Briefly explain the code flow for user list sorting
- Briefly explain the code flow for server-side to UI of turn indicators

Your Response:

The server updates a player's score and broadcasts a PointsPayload to all clients, which update their local user objects; the UI then refreshes the scoreboard table to show the new points. Each client sorts all known users by client ID before sending them to the UI, ensuring every client displays the list in the same consistent order. The server broadcasts whose turn it is, clients

update currentTurnClientId, and the UI highlights that player and updates the “Current Turn” label so all players see the same turn state.

Saved: 12/8/2025 3:27:15 PM

Task #2 (0.67 pts.) - Game Events Panel

Progress: 100%

Details:

- Show the letter/guess history (including points)
- Show the scoreboard updates from Milestone 2
- Show a message of whose turn it is
- Show the countdown timer for the current turn

Part 1:

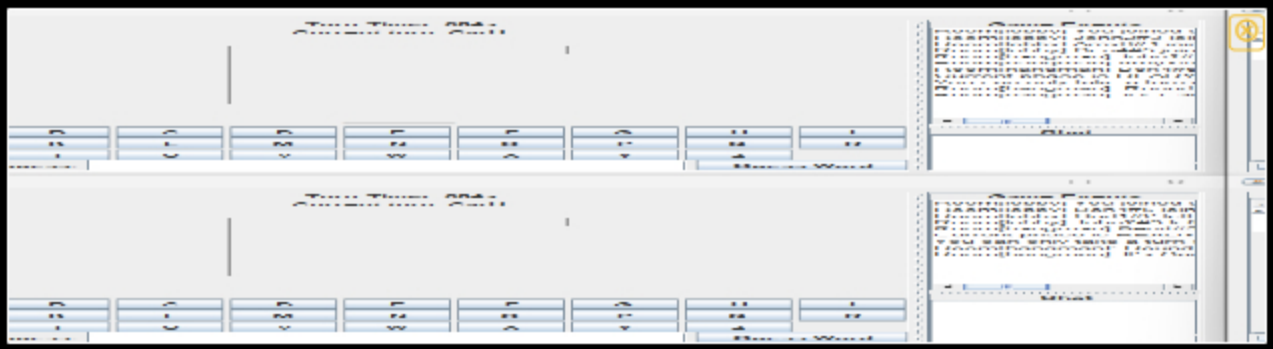
Progress: 100%

Details:

- Show various examples of each of the messages/visuals
- Show code snippets related to these messages from server-side to UI



example messages



countdown timer

Saved: 12/8/2025 5:39:04 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for generating these messages and getting them onto the UI

Your Response:

The server generates readable game messages and sends them to clients as payloads. The client receives these payloads, updates its internal state, and stores text messages in a queue. The UI timer polls this state, pulls new messages, and displays them in the Game Events panel while updating turn indicators and other visuals.

Saved: 12/8/2025 5:39:04 PM

Task #3 (0.67 pts.) - Game Area

Progress: 100%

Details:

- Have some elements representing the Hangman or equivalent (can't just be a number presenting strikes)
- Have a grid of letters that the user will click to interact with
  - Chosen letters should be synced to disable the option on all clients (via server-side reply)
  - Re-enable all letters when a hangman resets
- Have a special input area for guessing the full word (separate from the chat input)
- Display blanks for the word
  - These must update as letters are guessed correctly or word is fully guessed

Part 1:

Progress: 100%

Details:

- Show various examples of the hangman/strikes indicator across 3+ clients
  - Show the related code from server-side to UI
- Show various examples of selected letters and partially completed words across 3+ clients
  - Show the related code from UI to server-side and server-side to UI for both selection and blanks
- Show the related code for a guess (UI to server-side)

```
private void syncBlanksAndStrikes() {
    String blanks = new String(currentBlanks);
```

```
    clientsInRoom.values().forEach(st -> st.sendGameState(blanks, strikes, MAX_STRIKES));
}
```

strike indicator code snippet

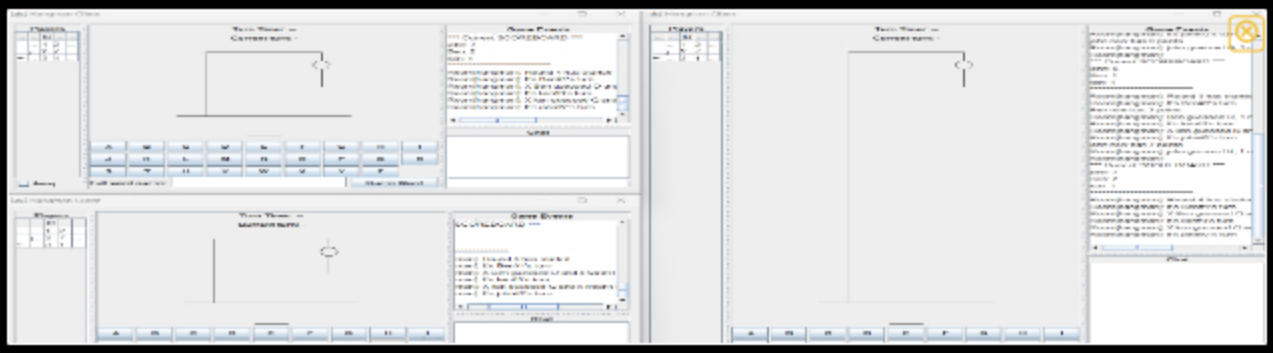
```
public void setStrikes(int current, int max) {
    this.currentStrikes = current;
    this.maxStrikes = max;
    repaint();
} <- #729-733 public void setStrikes(int current, int max)
```

strike code snippet UI

```
public void guessLetterFromUI(char c) {
    try {
        String letter = String.valueOf(Character.toUpperCase(c));
        sendDoTurn("/letter " + letter);
    } catch (IOException e) {
        LoggerUtil.INSTANCE.severe("Failed to send letter guess", e);
    }
} <- #xxx-xxx public void guessLetterFromUI(char c)

/**
 * Wrapper for UI to send a full word guess as a TURN.
 * Uses /guess <word> to match console behavior and server parsing.
 */
public void guessWordFromUI(String word) {
    try {
        if (word != null && !word.isBlank()) {
            sendDoTurn("/guess " + word.trim());
        }
    } catch (IOException e) {
        LoggerUtil.INSTANCE.severe("Failed to send word guess", e);
    }
} <- #211-219 public void guessWordFromUI(String word)
```

guess code snippet



strike sample

Saved: 12/8/2025 7:46:10 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for syncing the strikes and displaying them visually
- Briefly explain the code flow for letter selection and disabling (including updates to the blanks)
- Briefly explain the code flow for a full word guess

- Briefly explain the code for a full word guess

Your Response:

Strikes: When an incorrect guess is made, the server increments the strikes counter and calls syncBlanksAndStrikes() which sends a GameStatePayload to all clients; the UI receives this, passes it to HangmanPanel.setStrikes(), and paintComponent() calculates and draws the appropriate number of body parts (0-6) based on the strikes ratio. Letter Selection: A clicked letter button triggers client.guessLetterFromUI() which sends /letter to the server; the server's processLetter() checks for matches in currentWord, updates currentBlanks array positions, awards points, and syncs the new state back to clients; the UI displays the updated blanks and disables the button (except in hard mode). Word Guess: Typing a word and clicking the guess button sends /guess to the server; processGuess() compares it to currentWord and either reveals the entire word by setting currentBlanks = currentWord.toCharArray() and ending the round (if correct), or increments strikes and continues play (if incorrect); all clients receive the updated state via GameStatePayload.

Saved: 12/8/2025 7:46:10 PM

Section #3: (4 pts.) Project Extra Features

Progress: 100%

Task #1 (2 pts.) - Restore Strikes

Progress: 100%

Details:

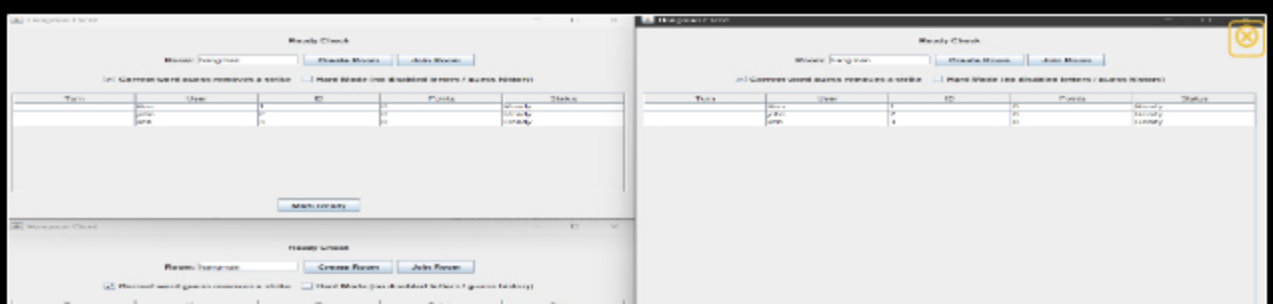
- Setting should be toggleable during Ready Check by session creator
- Allow toggling of the option to let correct guesses restore strikes

Part 1:

Progress: 100%

Details:

- Show the Ready Check screen with the option for the host (3+ clients must be visible)
  - Show the related code that makes this interactable only for the host
- Show a before and after of a strike being restored
  - Show the related code for the UI and handling this



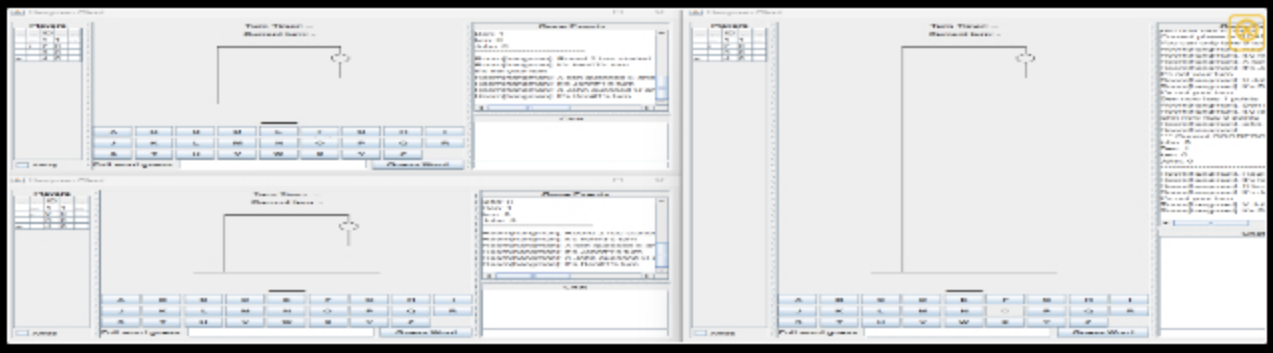


ready check screen

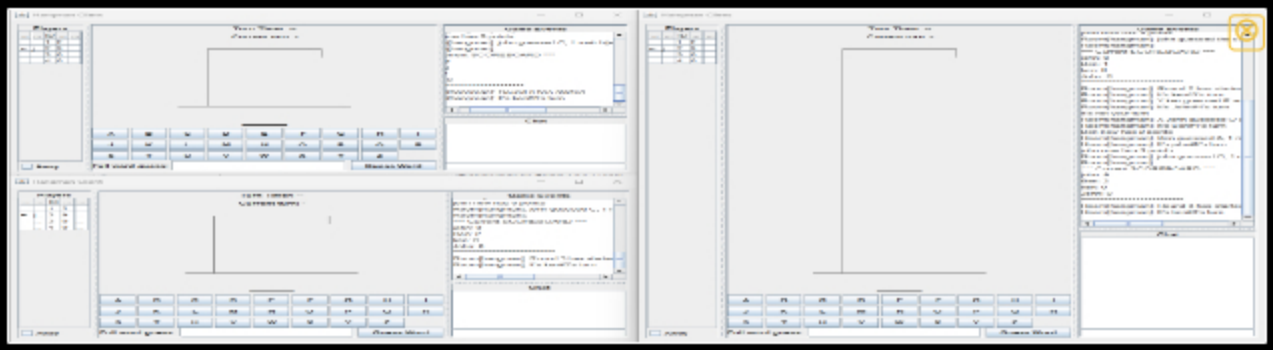
```
JPanel bottom = new JPanel(new FlowLayout(FlowLayout.CENTER));
bottom.add(readyButton);
add(bottom, BorderLayout.SOUTH);

// ---- Button actions ----
```

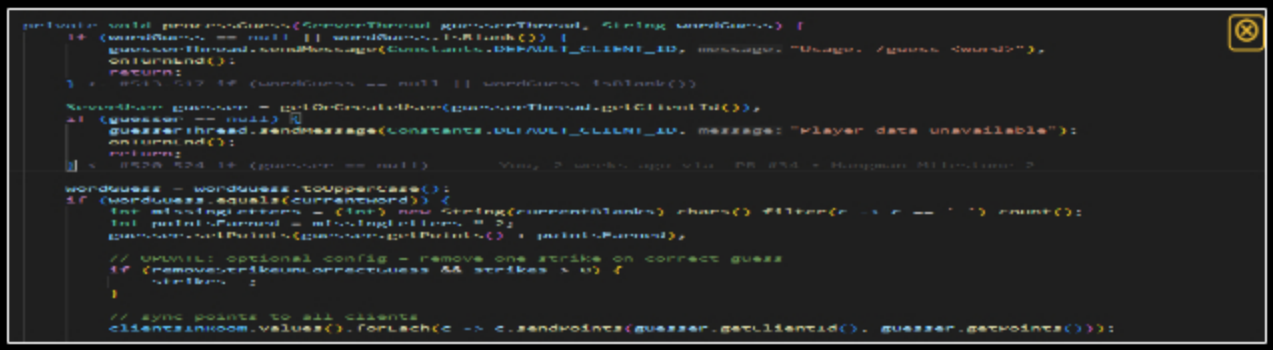
ready check code snippet



before strike restore



after strike restore



code snippet for restore strike

```
        });

        readyButton.addActionListener(e -> {
            removeStrikeOnCorrectGuess = removeStrikeCheck.isSelected();
```

```
            hardMode = hardModeCheck.isSelected();
            client.sendConfigFromUI(removeStrikeOnCorrectGuess, hardMode);
            client.toggleReadyFromUI();
        }); <- #414-419 readyButton.addActionListener
    } <- #357-420 ReadyPanel()
```

code snippet for restore strike

Saved: 12/8/2025 8:13:08 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code for the host's option to toggle this feature
- Briefly explain the code related to handling the strike restoration (UI and server-side)

Your Response:

When a player correctly guesses the full word, the server's processGuess() checks if (removeStrikeOnCorrectGuess && strikes > 0) and if true, executes strikes-- to decrement the counter, then calls syncBlanksAndStrikes() which sends a GameStatePayload with the updated strike count to all clients. The client receives this payload in processGameState(), stores it in latestGameState, and the UI's pollClientState() method retrieves it and calls gamePanel.updateGameState() which passes the strikes to hangmanPanel.setStrikes(). The HangmanPanel.paintComponent() calculates the ratio (currentStrikes / maxStrikes) * 6 to determine how many body parts to draw, so when strikes decreases from 3 to 2, the visual automatically removes one body part from the gallows display.

Saved: 12/8/2025 8:13:08 PM

Task #2 (2 pts.) - Hard Mode

Progress: 100%

Details:

- Setting should be toggleable during Ready Check by session creator
- Hard mode prevents the letter buttons from disabling and won't show guessed letters in the Game Event Area

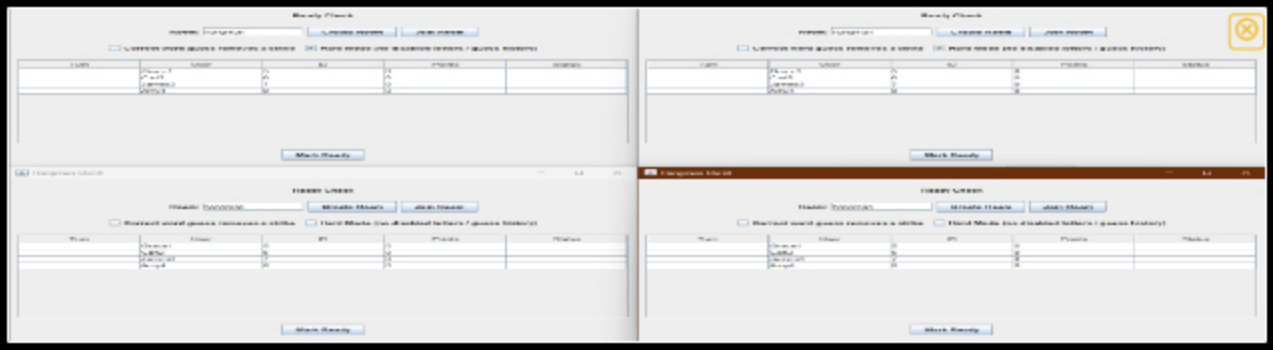
Part 1:

Progress: 100%

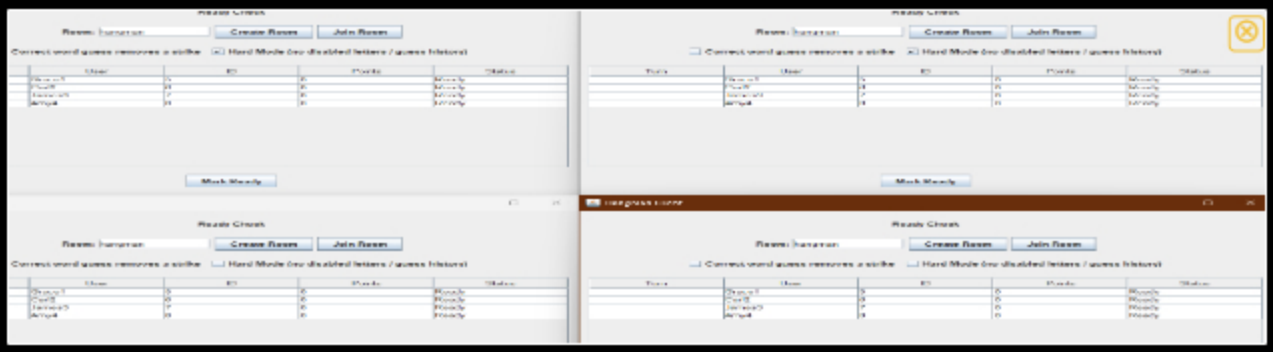
Details:

- Show the Ready Check screen with the option for the host (3+ clients must be visible)
    - Show the related code that makes this interactable only for the host

- Show a few examples of the play screen with a few turns to show these visuals have been disabled
    - Show the related code for the UI and handling the logic for hard mode



Hard mode toggle



ready check

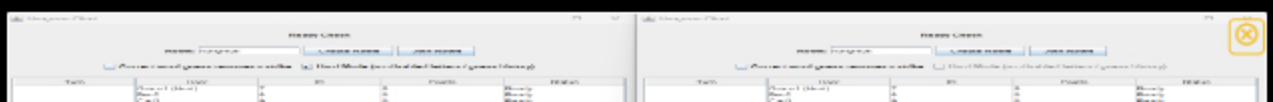
```java
public boolean isHost() {
    if (knownClients.isEmpty()) {
        return false;
    }
    Long minId = knownClients.keySet().stream()
            .min(Long::compareTo)
            .orElse(myUser.getClientId());
    return myUser.getClientId() == minId;
} <- #110-118 public boolean isHost()

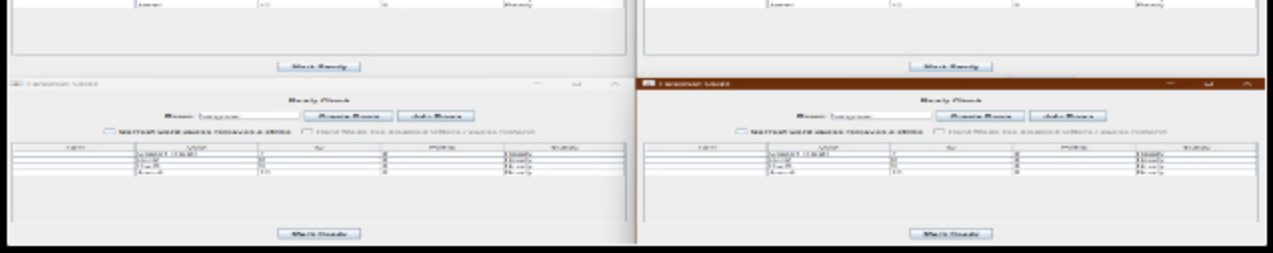
/**
 * Helper for UI to know if this user is currently a spectator.
 * (Value is driven by the server via User flags.)
 */
public boolean isSpectator() {
    return myUser.isSpectator();
}
```

host only toggle

```java
// host-only Hard Mode toggle
readyPanel.updateHostControls(client.isHost());
```

Host only





host sample

Saved: 12/8/2025 10:56:04 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code for the host's option to toggle this feature
- Briefly explain the code related to handling and enforcing this feature

Your Response:

Toggling Hard Mode makes the host enables the checkbox, the UI sends a /config TURN payload, and the server updates hardMode and broadcasts it to all players.

Saved: 12/8/2025 10:56:04 PM

Section #4: (2 pts.) Project General Requirements

Progress: 100%

Task #1 (1 pt.) - Away Status

Progress: 100%

Details:

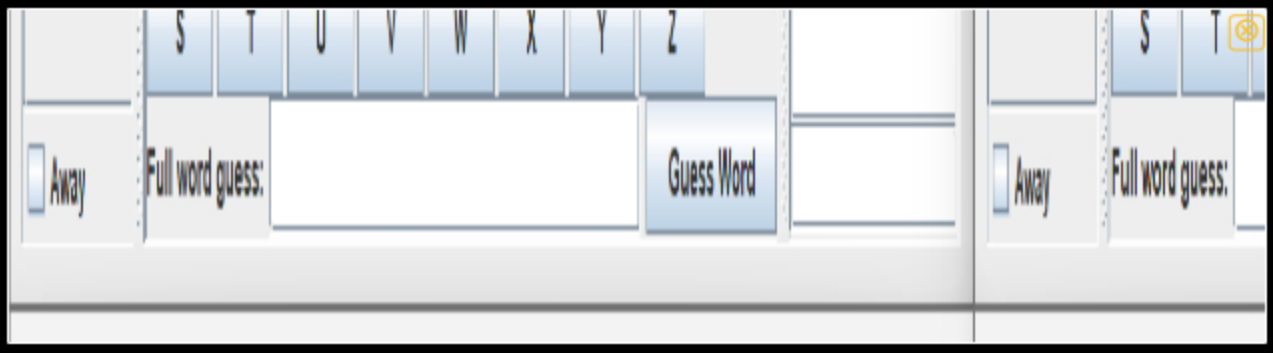
- Clients can mark themselves away and be skipped in turn flow but still part of the game
- The status should be visible to all participants
- A message should be relayed to the Game Events Panel (i.e., Bob is away or Bob is no longer away)
- The user list should have a visual representation (i.e., grayed out or similar)

Part 1:

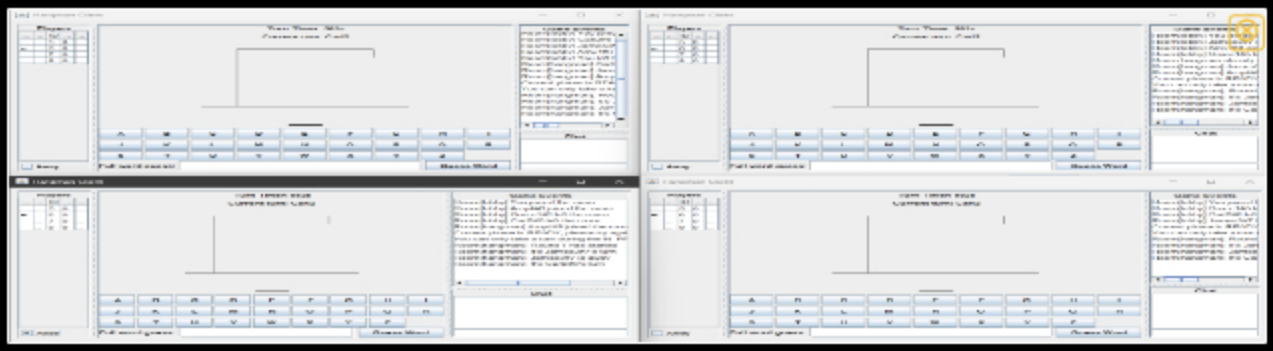
Progress: 100%

Details:

- Show the UI button to toggle away
- Show the related code flow from UI to server-side back to UI for showing the status
- Show the related code flow for sending the message to Game Events Panel
- Show various examples across 3+ clients of away status (including Game Events Panel messages)
- Show the code that ignores an away user from turn/round logic



Away button



Is away examples



code flow for game event messages

Saved: 12/8/2025 8:37:57 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for the away action from UI to server-side and back to UI
- Briefly explain how the server-side ignores the user from turn/round logic

Your Response:

The server's turn selection method (getNextPlayer()) loops through the turn order.

For each candidate user, it checks SeverUser.isAway().

If the user is Away, they are skipped, and the loop moves to the next player.

Only the first non-away user is selected as the current turn player, ensuring Away players never receive a turn until they return.

Saved: 12/8/2025 8:37:57 PM

Task #2 (1 pt.) - Spectators

Progress: 100%

Details:

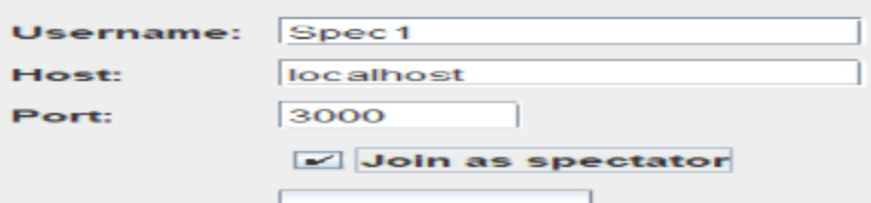
- Spectators are users who didn't mark themselves ready
  - Optionally you can include a toggle on the Ready Check page
- The can see all chat but are ignored from turn/round actions and can't send messages
- Spectators will have a visual representation in the user list to distinguish them from other players
- A message should be relayed to the Game Events Panel that a spectator joined (i.e., during an in-progress session)

Part 1:

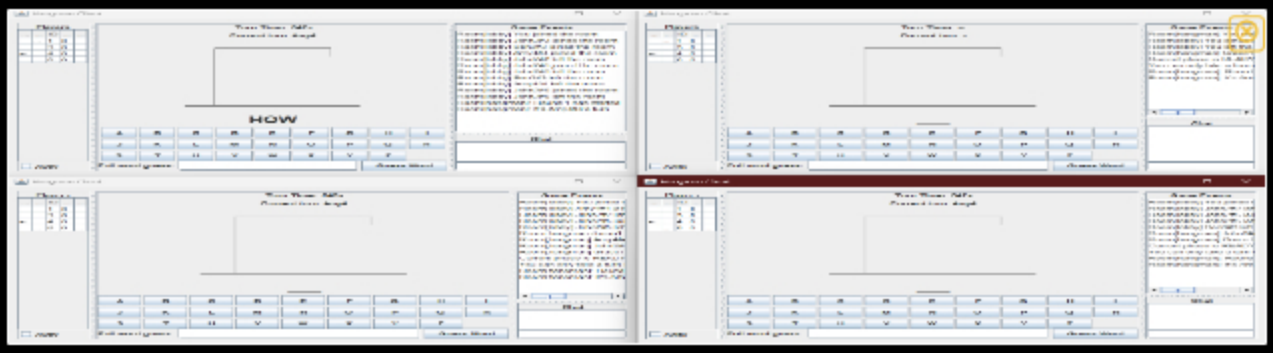
Progress: 100%

Details:

- Show the UI indicator of a spectator (visual and message)
- Show the related code flow from UI to server-side back to UI for showing the status
- Show the related code flow for sending the message to Game Events Panel
- Show various examples across 3+ clients of spectator status (including Game Events Panel messages)
- Show the code that ignores a spectator from turn/round logic
- Show the code that prevents spectators from sending messages (server-side)
- Show the spectator's view of the session
- Show the code related to the spectator seeing the session data (including things participants won't see such as the correct word)



Spectator



can see the correct word

```
protected void checkIsReady(ServerThread client) throws NotReadyException {
    if (!client.isReady()) {
        client.sendMessage(Constants.DEFAULT_CLIENT_ID, message: "You must be marked 'ready' to do this action");
        throw new NotReadyException(message: "Not ready");
    }
} <- #280-285 protected void checkIsReady(ServerThread client) throws NotRe...
  You, 3 weeks ago via  PR #33 • Turn based code
```

ignores a spectator from turns

```
    case TURN:
        // no data needed as the intent will be used as the trigger
        try {
            // cast to GameRoom as the subclass will handle all Game logic
            ((GameRoom) currentRoom).handleTurnAction(this, incoming.getMessage());
        } catch (Exception e) {
            sendMessage(Constants.DEFAULT_CLIENT_ID, message: "You must be in a GameRoom to do a turn");
        }
        break;
    default:
        LoggerUtil.INSTANCE.warning(TextFX.colorize(text: "Unknown payload type received", Color.RED));
        break;
} <- #224-271 switch (incoming.getPayloadType())
#222-272 protected void processPayload(Payload incoming)
```

prevents spectators from sending messages

```
private void syncBlanksAndStrikes() {        You, 2 weeks ago via  PR #
String blanks = new String(currentBlanks);

clientsInRoom.values().forEach(st -> {
    // Players (ready) see blanks, spectators see full word
    String wordForThisClient = st.isReady() ? blanks : currentWord;

    st.sendGameState(wordForThisClient, strikes, MAX_STRIKES);
}); <- #388-393 clientsInRoom.values().forEach
<- #385-394 private void syncBlanksAndStrikes()
```

Show the code related to the spectator seeing the session data (including things participants won't see such as the correct word

Saved: 12/8/2025 9:52:20 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for the spectator logic from server-side and to UI
- Briefly explain how the server-side ignores the user from turn/round logic
- Briefly explain the logic that prevents spectators from sending a message
- Briefly explain the logic that shares extra details to the spectator (information normal participants won't see)

Your Response:

The server decides whether a user is a player or spectator by checking isReady(), sending players the masked word and spectators the full word during syncBlanksAndStrikes(), which the UI simply displays without knowing the difference. Spectators are excluded from gameplay because they are never added to turnOrder, and any turn or action they attempt fails checkIsReady(), preventing them from influencing the round. The server also blocks spectator messages by rejecting their MESSAGE or TURN payloads and only delivers extra session details—like the correct word—when generating their individualized game state payload.

Saved: 12/8/2025 9:52:20 PM

Section #5: (1 pt.) Misc

Progress: 100%

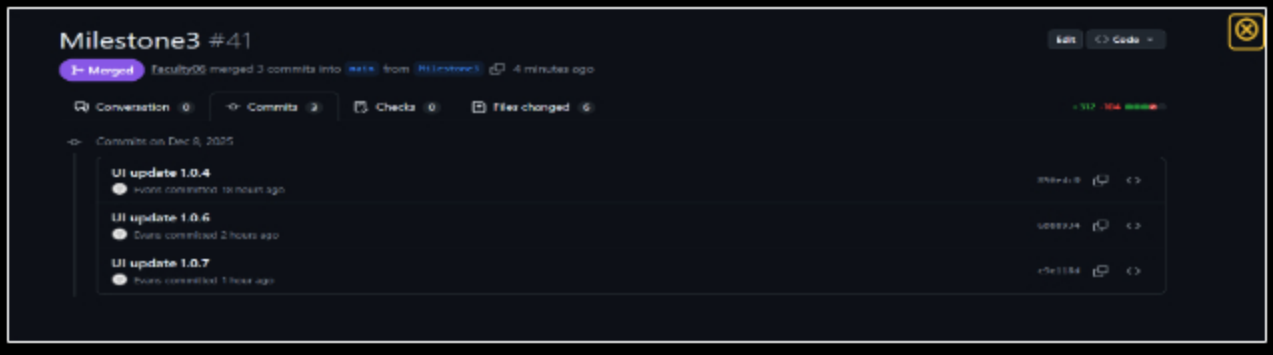
Task #1 (0.33 pts.) - Github Details

Progress: 100%

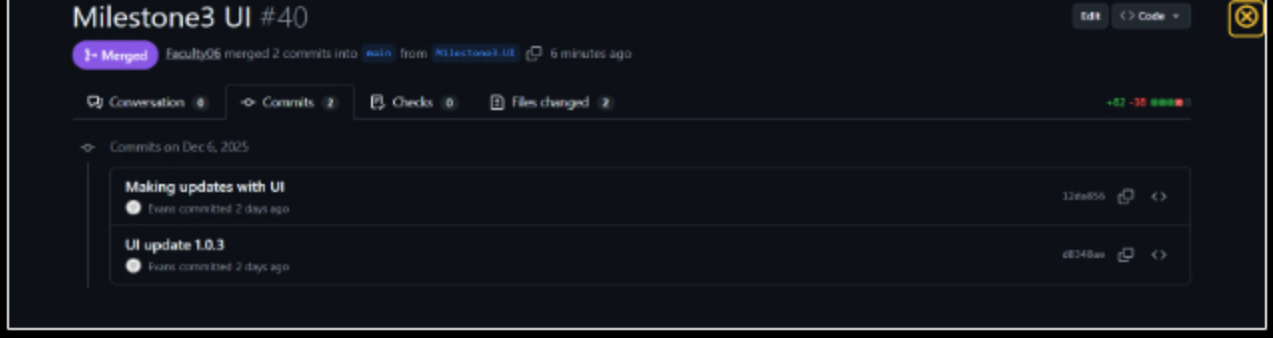
Part 1:

Progress: 100%

Details:
From the Commits tab of the Pull Request screenshot the commit history



commit history



commit history

Saved: 12/8/2025 7:22:35 PM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request for Milestone3 to main (should end in `/pull/#`)

URL #1
https://github.com/Faculty06/Ekn3-IT1[illegible]

URL
https://github.com/Faculty06/Ekn

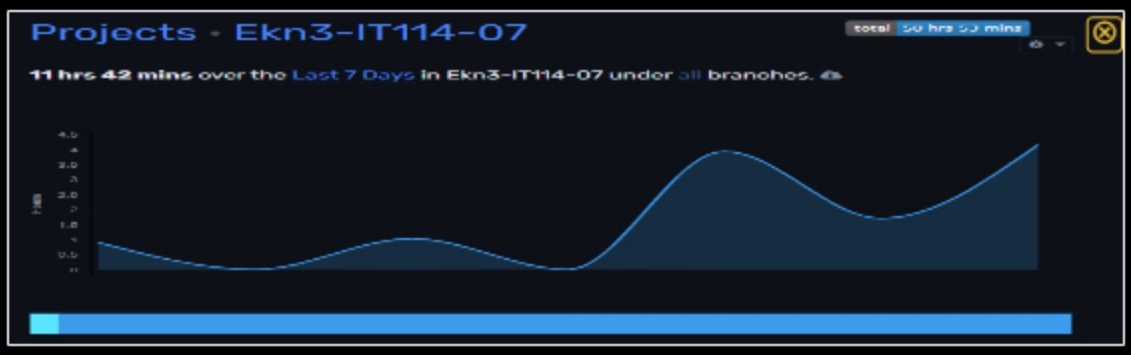
Saved: 12/8/2025 7:22:35 PM

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



waka time



WAKA TIME

Saved: 12/8/2025 7:14:29 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

Murphys law lol (Doing assignments on last days is a risky move)

How tabs hugely affects athe order of code.

Saved: 12/8/2025 6:31:13 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

Adding away and spectator toggles

Saved: 12/8/2025 6:26:58 PM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

Making turn timer sync on the UI

Saved: 12/8/2025 6:25:41 PM